Práctica DIAS
Desarrollo completo de un sistema software

1. Organización

La práctica se realizará en grupos de 3 estudiantes.

Cada grupo tendrá asignado un nombre que podrán ver en el campus virtual y deberá utilizarlo como identificador del repositorio y en la documentación del trabajo.

Ejemplos:

- alpha
- bravo
- ...

2. Objetivo

El objetivo de la práctica es desarrollar, de forma integrada, parte de un sistema software a partir de un documento de requisitos, recorriendo los workflows de:

- Análisis
- Diseño
- Implementación
- Pruebas

El trabajo debe mostrar coherencia y trazabilidad entre todos estos niveles.

3. Material proporcionado

Se proporcionará un documento completo de requisitos que servirá como base del trabajo.

4. Alcance de la práctica

NO se debe implementar el sistema completo.

Cada grupo trabajará únicamente sobre un subconjunto acotado de funcionalidad del sistema clínico descrito en el enunciado.

El objetivo de la práctica es desarrollar correctamente los principales artefactos del proceso de desarrollo de software, abarcando las fases de análisis, diseño e implementación de forma coherente.

4.1. Funcionalidad obligatoria

En esta práctica se desarrollará:

- El modelo de dominio completo del sistema, identificando las clases conceptuales, sus atributos y relaciones.
- La realización de los siete primeros casos de uso definidos en el enunciado (CU01–CU07), mediante diagramas de análisis.

4.2. Casos de uso a desarrollar

Se deberán desarrollar los siguientes casos de uso del sistema:

- CU01 – Iniciar sesión
- CU02 – Registrar paciente
- CU03 – Crear procedimiento clínico
- CU04 – Registrar informe clínico inicial
- CU05 – Registrar resultados de analítica
- CU06 – Consultar procedimiento clínico
- CU07 – Dar de alta al paciente

4.3. Alcance del diseño

- Definición de la arquitectura lógica del sistema mediante un diagrama de paquetes o capas.
- Descripción de las responsabilidades de cada subsistema o capa.
- Realización en diseño de los casos de uso indicados por el profesor, mediante diagramas de clases y de secuencia.

4.4. Alcance de la implementación

- Implementación parcial del sistema, centrada en los casos de uso seleccionados.
- Organización del código conforme a la arquitectura definida en el diseño.
- Validación básica del funcionamiento del sistema.

4.5. Alcance y limitaciones

- El trabajo se desarrollará principalmente en el ámbito del **análisis**, evolucionando de forma progresiva y coherente hacia las fases de diseño y una implementación parcial de los casos de uso propuestos. Dicha implementación deberá exponerse mediante una API REST desarrollada con Flask, incorporando persistencia en un sistema gestor de bases de datos relacional distinto de SQLite, como *MySQL* o *PostgreSQL*, a elegir por los miembros del grupo. Asimismo, la implementación deberá incluir obligatoriamente los mecanismos necesarios para permitir la ejecución de pruebas con pytest mediante persistencia en memoria, así como los recursos y configuraciones requeridos para la validación de los casos de uso utilizando la herramienta *Bruno*.

- No se requiere implementar la totalidad del sistema, sino únicamente un subconjunto representativo, es decir, los 7 casos de uso listados anteriormente.
- En el caso de uso de iniciar sesión, se recomienda usar JWT (https://www.jwt.io/). Podéis encontrar módulos Python aquí (https://www.jwt.io/libraries?programming_language=python)
- El modelo debe centrarse en el **dominio del problema**, representando los conceptos clínicos y sus relaciones.
- En la realización de casos de uso se deben identificar las clases *boundary*, *control* y *entity*, así como su interacción mediante diagramas de secuencia.
- Los diagramas deben centrarse en el comportamiento esencial de cada caso de uso, evitando detalles innecesarios o de bajo nivel.
- No es necesario modelar todos los escenarios alternativos, salvo aquellos relevantes para comprender el flujo principal.

El objetivo es obtener una solución coherente, bien estructurada y correctamente modelada, manteniendo la trazabilidad entre análisis, diseño e implementación.

5. Repositorio

El trabajo se desarrollará en un repositorio GitLab creado por el portavoz del grupo.

5.1. Nombre recomendado

`dias2526-<grupo>`
El nombre del grupo (en minúsculas) lo podéis ver en el campus virtual.
Ejemplos:

- `dias2526-alpha`
- `dias2526-bravo`

6. Estructura recomendada del repositorio

La estructura es muy similar, por no decir igual, a la planteada en las sesiones individuales. Esta es una recomendación, pero podéis adaptarla de forma justificada.

```
/
|-- README.md
|-- docs/
|   |-- requisitos/
|   |   \-- enunciado.pdf
|   \-- diagramas/
|       |-- dominio.png
|       |-- analisis_secuencia.png
|       |-- diseno_clases.png
|       \-- arquitectura.png
```

```
|-- src/
|   |-- domain/
|   |-- application/
|   |-- infrastructure/
|   \-- main.py
|-- bruno/
|   |-- # El entorno de Bruno para ejecutar los casos de uso
|-- tests/
|   |-- test_xxx.py
|   \-- ...
|-- pyproject.toml
|-- .gitlab-ci.yml # Opcional
\-- .gitignore
```

Nota: Recordar dar permisos al profesor (perfil @javalon) para visualizar el repositorio.

7. Recomendaciones de uso de Git y GitLab

El repositorio no debe utilizarse únicamente como mecanismo de entrega, sino como herramienta de trabajo colaborativo durante todo el desarrollo de la práctica. Se recomienda que el grupo adopte una dinámica de trabajo basada en ramas, *merge requests*, revisiones entre compañeros y mensajes de commit claros y consistentes.

7.1. Trabajo con ramas

Se recomienda no trabajar directamente sobre la rama principal (`main` o `master`). Cada cambio relevante debería desarrollarse en una rama específica, creada a partir de la rama principal.

Conviene utilizar nombres de rama descriptivos y homogéneos. Por ejemplo:

- `feature/cu01-login`
- `feature/modelo-dominio`
- `feature/api-pacientes`
- `fix/validacion-fecha`
- `docs/readme`
- `test/cu03-procedimiento`

La idea es que cada rama represente una tarea o cambio acotado, de forma que su propósito sea fácil de entender.

7.2. Uso de *merge requests*

La integración del trabajo en la rama principal debería realizarse preferentemente mediante *merge requests* en GitLab. No se recomienda hacer *push* directo a la rama principal salvo en casos excepcionales y justificados.

Cada *merge request* debería:

- tener un título claro y descriptivo;
- agrupar cambios relacionados entre sí;
- incluir una breve explicación de lo realizado;
- permitir que otro integrante del grupo revise el trabajo antes de integrarlo.

Siempre que sea posible, se recomienda que las *merge requests* sean pequeñas y frecuentes, en lugar de acumular muchos cambios en una única integración final.

7.3. Revisión del trabajo

El uso de GitLab debe reflejar una colaboración real entre los miembros del grupo. Por ello, se recomienda revisar el trabajo de los compañeros antes de aceptar una *merge request*, comprobando al menos:

- que el cambio tiene sentido respecto al caso de uso o artefacto trabajado;
- que no rompe la estructura general del proyecto;
- que mantiene la coherencia con el análisis, el diseño y la implementación;
- que la documentación y los nombres utilizados son comprensibles.

No se busca una revisión exhaustiva como en un entorno profesional, pero sí una evidencia mínima de trabajo coordinado y validación entre iguales.

7.4. Mensajes de commit

Los commits deben ser frecuentes, pequeños y con mensajes significativos. Deben evitarse mensajes genéricos como:

- `cambios`
- `arreglo`
- `update`
- `cosas`

Se recomienda utilizar el formato de *Conventional Commits*, que ayuda a expresar de forma clara el propósito de cada commit. Ejemplos válidos:

- `feat: add patient registration endpoint`
- `fix: validate discharge date correctly`
- `docs: update README with traceability table`
- `test: add unit tests for login use case`
- `refactor: move persistence logic to repository`

De forma orientativa, pueden utilizarse los siguientes prefijos:

- `feat`: nueva funcionalidad;
- `fix`: corrección de errores;
- `docs`: cambios en documentación;
- `test`: incorporación o modificación de pruebas;
- `refactor`: reorganización interna del código sin cambiar el comportamiento esperado;
- `chore`: tareas auxiliares de configuración o mantenimiento.

7.5. Frecuencia y continuidad del trabajo

El repositorio debe reflejar trabajo continuado a lo largo de la práctica. No se recomienda concentrar toda la actividad en los últimos días. Se valorará positivamente que el historial muestre una evolución progresiva del proyecto: definición del dominio, análisis, diseño, implementación, pruebas y documentación.

7.6. Relación entre Git y la evaluación

El uso adecuado de Git y GitLab permitirá observar mejor la participación de los integrantes del grupo y la forma en que se ha organizado el trabajo. En particular, se tendrá en cuenta que el repositorio muestre, de forma razonable:

- uso de ramas para desarrollar tareas concretas;
- integración mediante *merge requests*;
- mensajes de commit claros y consistentes;
- actividad repartida a lo largo del tiempo;
- colaboración real entre los miembros del grupo.

No se evaluará únicamente la cantidad de commits, sino la calidad del uso del repositorio como herramienta de desarrollo colaborativo.

8. README como memoria

El archivo `README.md` será el documento principal de la práctica. En un TFG real se recomienda el uso de LaTex, pero en esta asignatura, usaremos Markdown.

Debe permitir entender el trabajo completo sin necesidad de revisar todo el código.

9. Contenido del README

El README deberá seguir al menos la siguiente plantilla:

```
# Práctica DIAS - Grupo <NOMBRE_GRUPO>

## Integrantes
- Nombre Apellido 1
- Nombre Apellido 2
- Nombre Apellido 3

## Repositorio
URL del repositorio GitLab

---

# 1. Introducción

Breve descripción del sistema y del objetivo de la práctica.

---

# 2. Alcance del trabajo

## Funcionalidad trabajada
- CU01 - Iniciar sesión
- CU02 - Registrar paciente
- ...

## Decisiones sobre el alcance funcional
Indicar qué aspectos de los casos de uso se han simplificado, omitido o adaptado durante el desa

---

# 3. Modelo de dominio

## 3.1 Clases identificadas
Descripción de las principales clases del dominio.

## 3.2 Diagrama de dominio
![Dominio](docs/diagramas/dominio.png)

## 3.3 Comentarios
Explicación de decisiones tomadas.

---
```

```
# 4. Modelo de análisis

## 4.1 Casos de uso
Descripción breve.

## 4.2 Clases de análisis
Explicación de su papel.

## 4.3 Diagramas de secuencia
![Secuencia](docs/diagramas/analisis_secuencia.png)

## 4.4 Comentarios
Decisiones importantes.

---

# 5. Diseño

## 5.1 Arquitectura
Describir la arquitectura del sistema basada en la separación en capas (domain, application, inf

- el papel de cada capa
- las responsabilidades principales
- cómo se relacionan entre sí

## 5.2 Diagrama de arquitectura
![Arquitectura](docs/diagramas/arquitectura.png)

## 5.3 Diseño de clases
![Clases](docs/diagramas/diseno_clases.png)

## 5.4 Decisiones de diseño
Describir las principales decisiones de diseño adoptadas, incluyendo:

- uso de patrones (si aplica)
- asignación de responsabilidades entre clases y capas
- organización del código

Por cada decisión relevante, justificar:

- qué problema resuelve
- cómo se relaciona con los casos de uso y el modelo de análisis
- qué impacto tiene en la implementación

---

# 6. Implementación
```

6.1 Estructura del código
Explicación de carpetas.

6.2 Tecnologías
Lenguaje, librerías, herramientas.

6.3 Descripción
Cómo se ha implementado la funcionalidad.

6.4 Ejecución

Instalación

```bash
poetry install
poetry run python -m app.main #Este es un ejemplo
```

7. Pruebas

7.1 Estrategia

Qué se prueba y por qué.

7.2 Tests implementados

Descripción breve.

7.3 Ejecución
```bash
poetry run pytest # Esto es un ejemplo
```

7.4 Cobertura (opcional)

8. Trazabilidad

Caso de uso	**Dominio**	**Análisis**	**Diseño**	**Código**	**Test**
CUxx	Clase X	Secuencia Y	Servicio Z	modulo.py	test_modulo.py

9. Planificación

Explicar cómo se ha organizado el trabajo durante las 8 semanas.

10. Conclusiones
 - Dificultades
 - Decisiones

```
 - Aprendizaje
 - Mejoras futuras

# Anexo I

## Videodefensa
<Enlace a la videodefensa>
```

10. Pruebas

La práctica debe incluir tests automáticos obligatorios.

10.1. Requisitos mínimos

- Tests unitarios sobre la lógica principal
- Ubicados en la carpeta `/tests`
- Poder lanzarse desde `Poetry` con `pytest`

10.2. Ejecución

Ejemplo:

```
poetry run pytest
```

o

```
poetry run test
```

11. Planificación orientativa (8 semanas)

Semanas 1–2

- Lectura de requisitos
- Definición de alcance
- Modelo de dominio

Semanas 3–4

- Modelo de análisis
- Diagramas de secuencia

Semanas 5–6

- Diseño del sistema
- Arquitectura y diagramas

Semanas 7–8

- Implementación
- Tests
- Documentación final

12. Reglas técnicas

12.1. Lenguaje

Python >= 3.12

12.2. Dependencias

Poetry >= 2.0

12.3. Tests

pytest

12.4. Arquitectura

Se recomienda separación por capas:

- domain
- application
- infrastructure

Esta arquitectura corresponde a la aplicación realizada de forma individual durante las 5 primeras sesiones de prácticas.

13. Videodefensa

Como parte de la evaluación de la práctica, cada grupo deberá realizar una videodefensa del trabajo desarrollado.

13.1. Formato

- Duración total: entre 10 y 12 minutos por grupo
- Participación obligatoria de todos los integrantes

Distribución orientativa:

- 6–8 minutos: exposición del trabajo
- 4–5 minutos: preguntas orientativas

13.2. Contenido de la exposición

Durante la videodefensa, el grupo deberá explicar:

- El alcance del sistema implementado
- El modelo de dominio propuesto
- Las principales decisiones de análisis
- La arquitectura y decisiones de diseño
- La estructura de la implementación
- La estrategia de pruebas
- Demostración básica de la implementación (si procede)

13.3. Preguntas orientativas

Durante la defensa, se debe responder a al menos 3 de las siguientes preguntas:

- ¿Por qué habéis elegido estas clases en el modelo de dominio?
- ¿Qué responsabilidades tiene esta clase y por qué?
- ¿Cómo se refleja este caso de uso en el diagrama de secuencia?
- ¿Qué cambios habría que hacer en el diseño si cambia un requisito?
- ¿Qué parte del código implementa este comportamiento?
- ¿Qué tipo de tests habéis implementado y qué validan?
- ¿Qué parte os ha resultado más compleja y por qué?
- Si tuvierais más tiempo, ¿qué mejoraríais?

13.4. Evaluación individual

La videodefensa permitirá evaluar la participación individual de cada integrante del grupo.

Se podrá ajustar la calificación individual en función de la participación y la comprensión demostrada durante la defensa.

14. Entrega de la videodefensa

Cada grupo deberá entregar una videodefensa del trabajo realizado.

14.1. Formato del vídeo

El vídeo deberá cumplir los siguientes requisitos:

- Formato: MP4 (H.264)
- Resolución recomendada: 720p (1280x720) o 1080p (1920x1080)
- Duración: entre 10 y 12 minutos
- Tamaño máximo recomendado: 200 MB

14.2. Compresión del vídeo

Para asegurar un tamaño adecuado del fichero, se recomienda utilizar la herramienta **HandBrake** https://handbrake.fr/.
Configuración recomendada:

- Preset: General → Fast 720p30 (o equivalente)
- Video Codec: H.264 (x264)
- Framerate: 30 fps (constant)
- Quality: RF 22–26 (mayor valor = menor tamaño)
- Audio: AAC, 128 kbps

El objetivo es obtener un vídeo con buena calidad, pero con un tamaño contenido.

14.3. Entrega

El vídeo no deberá subirse directamente al repositorio si supera los límites de tamaño.
Opciones de entrega:

- Subida a plataforma externa (YouTube, Google Drive, OneDrive, etc.)
- Inclusión del enlace en el README del repositorio

El enlace debe permitir la visualización por parte del profesorado.

14.4. Importante

- No se aceptarán vídeos en formatos no estándar
- No se aceptarán vídeos con tamaño excesivo (varios GB)
- El vídeo debe poder reproducirse sin necesidad de software especial

15. Importante

- No se evaluará positivamente hacer más funcionalidad si está mal hecha
- Se valorará la coherencia entre análisis, diseño y código
- Los tests son obligatorios
- El README debe ser claro y completo
- Los diagramas deben ser legibles
- El repositorio debe reflejar trabajo continuo con commits, merge-requests, etc.

16. Recomendación

No dejar la implementación y los tests para el final.
La coherencia entre modelos y código es parte esencial de la práctica.